통계학 정리본

💭 *통계학을 왜 배워야 하나?*

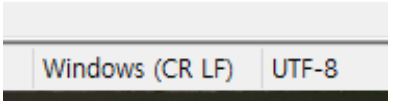
- 이 숫자는 **평범한가**, 아니면 **이상한가**
- 두 집단은 **정말 차이가 있는가**
- 지금 보이는 패턴이 **우연인가**, 아니면 **의미 있는 현상인가**
- 일부 데이터만 보고 전체를 **어느 정도 믿고 추정할 수 있는가**

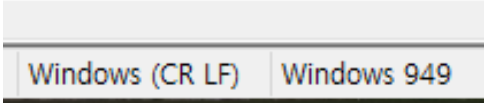
"데이터 속에 숨겨진 진실과 노이즈를 구분하기 위함이다."

UTF-8 vs CP949

외국 프로그램들은 utf-8이 기본 / 한국은 cp949(or euc-kr)가 기본

(이상하게 읽힌다면 각 상황에 맞게 encoding 해줘야 함.)





편차 / 분산 / 표준편차 정의

- 편차 : 평균에서 얼마나 떨어져있나 [m]
- 분산 : 편차 제곱의 평균 [m^2]
- 표준편차 : 분산의 루트 [m]

(편차와 표준편차는 비교 가능 / 분산은 단위 다르기 때문에 비교 불가)

데이터 전처리 3대장

항목	상태 (Problem)	해결 (Action)

1. 결측치	데이터에 **빈칸**이 있음	**삭제**가 원칙 (안 되면 채우기)
2. 이상치	혼자 너무 **튀는 값**	노이즈라면 **버리기**
3. 정규화	서로 다른 **단위/체급**	**0~1 사이**로 맞춰 공평하게 비교

💭 *pandas를 왜 배우는가?*

- CSV 파일
- 엑셀 파일
- DB에서 뽑은 표 형태 데이터
- 로그 데이터
- 설문 데이터

대부분의 데이터 파일들은 **행(row)과 열(column)** 구조이다.

Pandas는 바로 이 **표 형태 데이터**를 다루는 데 특화되어 있음.

쉽게 말하면:

- 엑셀보다 자동화가 쉽고
- SQL보다 파이썬 코드와 연결이 쉽고
- 분석/시각화/통계 처리로 바로 이어짐

---

pandas 명령어 (pandas = 파이썬에서 쓰는 데이터 엑셀)

Pandas는 처음에 딱 **2개**만 잡으면 된다.

1) Series

한 개의 열 같은 구조

```
import pandas as pd

s = pd.Series([10, 20, 30, 40])
print(s)
```

---

2) DataFrame

여러 개의 열이 모인 표 구조

```python
import pandas as pd

df = pd.DataFrame({
    "name": ["Kim", "Lee", "Park"],
    "age": [20, 22, 21],
    "score": [85, 90, 78]
})

print(df)
```

위 데이터를 표로 보면 아래와 같다.

name	age	score
Kim	20	85
Lee	22	90
Park	21	78

👉 행 = 샘플 (Kim, Lee, Park) / 열 = 속성 (name, age, score)

---

3) CSV 파일 불러오기

```python
import pandas as pd

df = pd.read_csv("sales.csv")
print(df)
```

대부분의 분석은 이 흐름으로 시작함.

1. 파일 불러오기 (위 코드)
2. 데이터 상태 확인
3. 정제
4. 분석
5. 시각화
6. 리포트

---

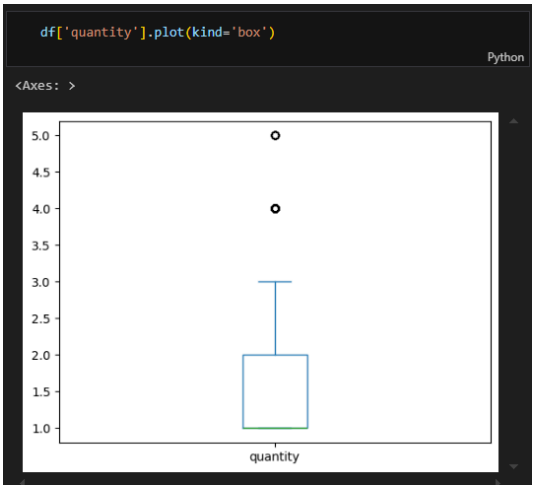
4) 데이터를 처음 받으면 해야할 5가지

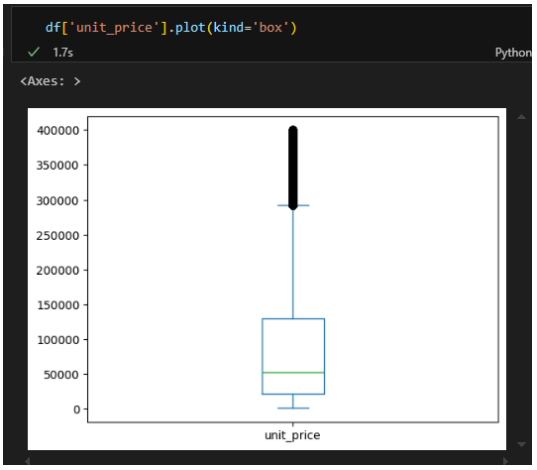
```
df.head(5)  # 첫 5행을 보여줌 / 컬럼명이 맞는지, 값이 이상하지 않은지 확인용

df.info()  # 데이터 구조 보기 / 행,열  개수 . 데이터 타입 , 결측치 여부 확인용

df.describe()  # 기초 통계 보기 / 개수 , 평균 , 표준편차 , 최대, 최소값 . 사분위수 확인용

df.isnull().sum()  # 결측치 확인 / 각 컬럼별 빈칸 개수를 확인용

df.columns  # 컬럼명 확인용 / df["score"] 로 컬럼의 데이터만 뽑을 수도 있음.
```

▼ *matplotlib 설치 코드 (그림으로 표시해주는 파이썬 라이브러리)*

```
!pip install matplotlib  # jupyter lab에서 설치하려면 앞에 ! 붙이면 됨.
```

예시 이미지





컬럼 선택

컬럼(각종 센서들-온도,거리,속도,소리,등) 행으로 기록 / 항목(각 센서들의 값들) 열로 기록 하지만 전체 컬럼들을 사용하진 않음. 따라서 내가 필요한 컬럼들만 뽑아서 사용.

1) 하나 선택

```
df["score"]
```

2) 여러개 선택

```
df[["name","score","age"]]

df[ df['score'] > 85 ]  # 85점인 것들만 표로 만들어서 나타내줌.
```

- 실행 결과

```
print( df['score']
> 85 )
```

위 코드 출력하면 오른쪽과 같이
True/False로 나옴.

0	False
1	True
2	False
3	True
4	True
Name: score	dtype: bool

3) 심화 선택

```
## version 1
df[['age','name','score']][df['score'] > 85]


## version 2
cond = df['score'] > 85
df[['age','name','score']][cond]


## version 3
cond_score = df['score'] > 85
```

```
cond_age = df['age'] <= 21

df[ ['age','name','score'] ][ (cond_score) & (cond_age) ]
```

👍 웬만하면 하드코딩은 하지 않는게 좋음. 변수로 만들어서 수정하기 편하게 할 것.

🚫 and 아니고 **&** 임. 그리고 양옆은 **괄호**로 묶어야 함.